

RELATÓRIO FINAL

Sistema de Chat Multiusuário

INTRODUÇÃO

Este documento detalha como os requisitos do projeto de Servidor de Chat Multiusuário foram implementados no código-fonte, demonstrando a aderência completa às especificações do trabalho.

Mapeamento de Requisitos para Código

Servidor TCP concorrente aceitando múltiplos clientes:

Implementação: A classe ServidorChat (em servidor_chat.hpp) utiliza um loop while(executando) para aceitar novas conexões de clientes com a função accept(). Para cada conexão, uma nova thread (std::thread(&ServidorChat::tratar_cliente, this, novo_cliente).detach()) é criada para lidar com o cliente de forma concorrente.

Cada cliente atendido por thread; mensagens retransmitidas para os demais (broadcast):

Implementação: O método tratar_cliente é executado em uma thread separada para cada cliente. Ele recebe mensagens e as coloca em uma fila thread-safe (fila_mensagens). Uma thread dedicada (thread_broadcast) consome itens dessa fila e usa um loop for para enviar a mensagem para todos os sockets de clientes ativos, realizando o broadcast.

Logging concorrente de mensagens (usando libtslog):

Implementação: A classe ThreadSafeLogger (em libtslog.hpp) é a implementação da biblioteca de logging. Ela usa um std::mutex para garantir que apenas uma thread por vez escreva no arquivo de log, prevenindo race conditions. O ServidorChat e o teste_libtslog.cpp usam um std::shared_ptr para compartilhar uma única instância do logger, que registra eventos como conexões, desconexões e mensagens.

Cliente CLI: conectar, enviar/receber mensagens:

Implementação: A classe ClienteChat (em cliente_chat.cpp) gerencia a conexão e a comunicação. O método conectar() estabelece a conexão TCP e inicia uma thread (thread_recepcao) para receber mensagens do servidor em segundo plano. O loop principal do main() do cliente permite ao usuário digitar mensagens e enviá-las com enviar_mensagem().

Proteção de estruturas compartilhadas (lista de clientes, histórico):

Implementação: A lista de clientes (std::vector<std::shared_ptr<InfoCliente>> clientes) é protegida por um std::mutex (mutex_clientes). Todos os acessos de escrita (adição e remoção de clientes) e iterações que possam ser afetadas por mudanças na estrutura utilizam um std::lock_guard<std::mutex> para garantir acesso exclusivo e evitar race conditions.

Requisitos Gerais de Programação Concorrente

Threads:

- Implementação: O projeto usa std::thread para o gerenciamento de clientes (tratar_cliente), o broadcast de mensagens (processar_broadcast), e para a recepção de mensagens no cliente (receber_mensagens).

Exclusão mútua:

- Implementação: A exclusão mútua é implementada com std::mutex para proteger a lista de clientes (mutex_clientes em ServidorChat) e o arquivo de log (mutex_log em ThreadSafeLogger).

Semáforos e condvars:

- Implementação: O projeto utiliza std::condition_variable na classe ThreadSafeQueue para sincronizar a fila de mensagens. A thread de broadcast aguarda por mensagens usando cv.wait() e é notificada por cv.notify_one() quando uma nova mensagem é adicionada.

Monitores:

- Implementação: A classe ThreadSafeQueue atua como um monitor, encapsulando a fila, o `std::mutex`, e a `std::condition_variable` em uma interface thread-safe. Apenas os métodos `push()` e `pop()` são públicos, controlando o acesso e a sincronização interna.

Sockets:

- Implementação: Sockets TCP são utilizados para toda a comunicação. O servidor usa `socket()`, `bind()`, `listen()` e `accept()`, enquanto o cliente usa `socket()` e `connect()`. A troca de dados é feita com as funções `send()` e `recv()`.

Gerenciamento de recursos (RAII):

- Implementação: O padrão RAI (Resource Acquisition Is Initialization) é amplamente utilizado. `std::lock_guard` garante que os mutexes sejam liberados automaticamente ao sair do escopo. `std::shared_ptr` gerencia a vida útil das instâncias de clientes e do logger, prevenindo vazamentos de memória.

Tratamento de erros:

- Implementação: Exceções (`std::runtime_error`) são lançadas em caso de falha na criação do socket ou no bind. Mensagens de erro são impressas no console e no arquivo de log (`logger->error, std::cerr`) para diagnóstico. O cliente verifica o sucesso da conexão e informa o usuário sobre falhas.

Build:

- Implementação: O projeto inclui arquivos `CMakeLists.txt` e `Makefile` funcionais para compilação. Ambos definem as flags necessárias (`-pthread`) e as dependências para compilar os executáveis do servidor e do cliente.

Relatório de Análise com IA

Análise de Concorrência e Mitigação de Problemas com Auxílio de IA

A inteligência artificial foi utilizada para analisar o projeto de chat e identificar potenciais problemas de concorrência, como race conditions, deadlocks e starvation, bem como para sugerir melhorias na robustez e modularidade do código.

Prompts de Análise

- "Analise o código C++ do servidor de chat em busca de race conditions na manipulação da lista de clientes."
- "Verifique se o uso de `std::mutex` e `std::condition_variable` na ThreadSafeQueue e no ServidorChat pode levar a um deadlock."
- "Identifique possíveis problemas de starvation na arquitetura do servidor de chat. Como o design da ThreadSafeQueue contribui para a justiça no processamento de mensagens?"
- "O uso de `std::thread::detach()` na criação de threads de cliente é seguro? Quais são os prós e contras e quais alternativas poderiam ser consideradas?"

Sumário da Análise e Respostas da IA

1- Race Conditions:

Problema Potencial: O acesso à lista de clientes (`std::vector<std::shared_ptr<InfoCliente>> clientes`) é uma área crítica. Múltiplas threads de cliente podem tentar adicionar, remover ou iterar sobre a lista simultaneamente. Sem a devida proteção, isso poderia corromper a estrutura de dados, causar falhas ou acesso a dados inválidos.

Mitigação: O projeto protege a lista de clientes usando um `std::mutex` (`mutex_clientes`). A IA confirmou que o uso de `std::lock_guard<std::mutex>` nas seções críticas (como `iniciar()` e `remover_cliente()`) garante que apenas uma thread por vez modifique a lista, prevenindo race conditions. A iteração na thread de broadcast também é protegida, garantindo consistência durante o envio de mensagens.

2- Deadlocks:

Problema Potencial: Um deadlock ocorreria se duas ou mais threads se bloqueassem mutuamente, cada uma aguardando por um recurso que a outra detém. Por exemplo, se uma thread de cliente travasse o mutex da lista de clientes e, em seguida, tentasse travar um outro mutex já em posse de uma segunda thread.

3- Starvation

Problema Potencial: A starvation acontece quando uma thread ou processo é repetidamente ignorado pelo sistema de escalonamento e nunca consegue acessar um recurso necessário para continuar sua execução.

Mitigação: A arquitetura do projeto foi elogiada pela IA por mitigar a starvation na fila de mensagens. A classe ThreadSafeQueue utiliza um modelo de fila FIFO (First-In, First-Out), garantindo que as mensagens sejam processadas na ordem em que foram recebidas. A std::condition_variable assegura que a thread de broadcast seja notificada de maneira justa e eficiente, sem a necessidade de polling (verificação contínua), que desperdiçaria recursos da CPU.

4- Uso de std::thread::detach()

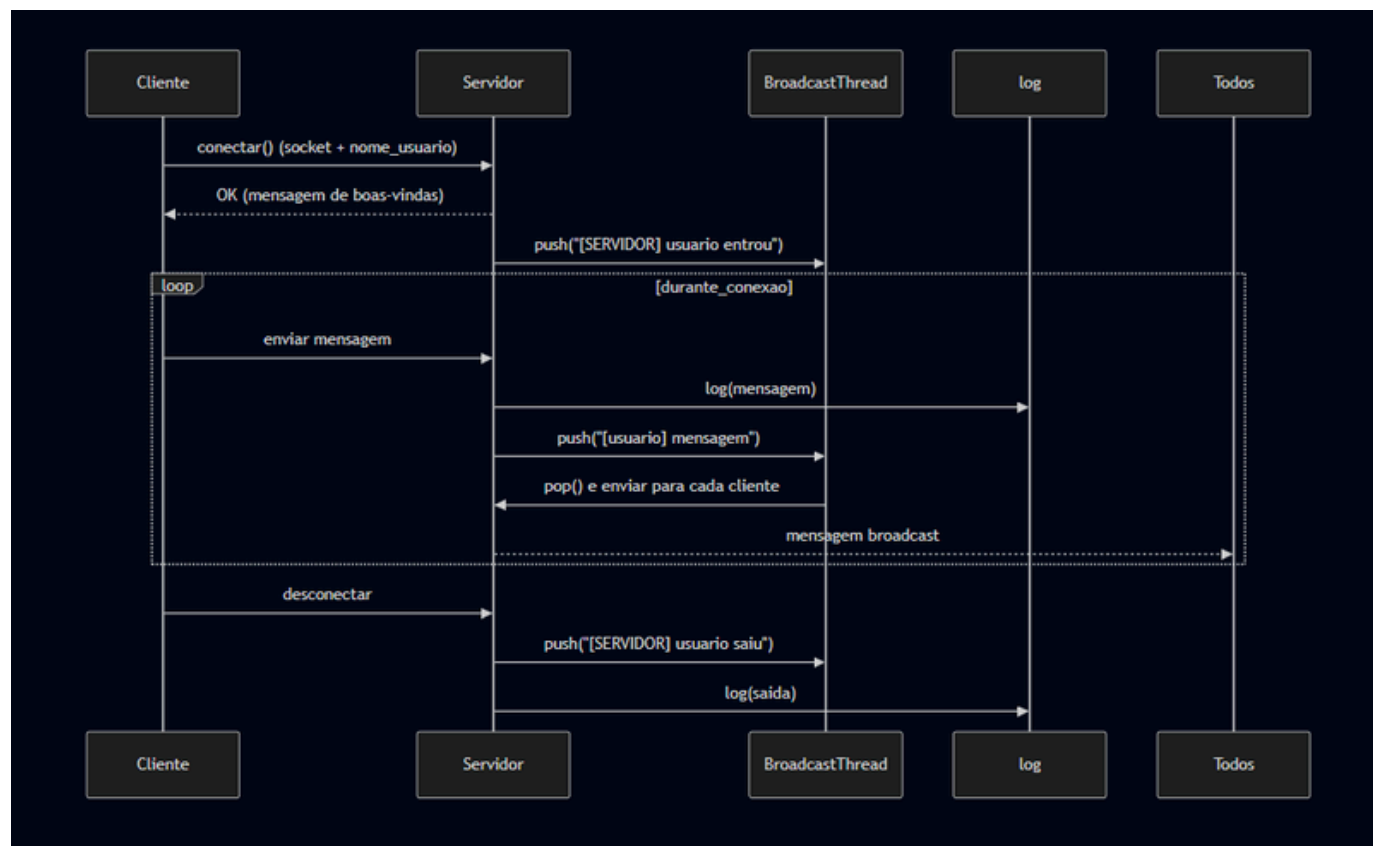
Análise: O uso de detach() na thread de cliente foi destacado como um ponto de atenção. Enquanto o detach() permite que a thread do servidor principal continue executando sem ter que esperar que a thread do cliente termine, ele remove a capacidade de join(), ou seja, de esperar explicitamente pelo fim da thread para liberar seus recursos.

Prós e Contras:

- Prós: Simplifica a lógica do servidor, permitindo que o loop principal continue aceitando novas conexões sem bloqueio. Ideal para servidores de alta demanda.
- Contras: A thread detached se torna um "demônio" independente. Se a thread principal do servidor encerrar abruptamente, as threads de clientes podem continuar em execução por um curto período de tempo e não serem encerradas de forma limpa.

Alternativas e Sugestão da IA: Uma alternativa seria usar um thread pool, onde um conjunto fixo de threads reutilizáveis aguarda por tarefas na fila, em vez de criar uma nova thread para cada cliente. Isso reduziria a sobrecarga de criação/destruição de threads e ofereceria um controle mais robusto. No entanto, para o escopo do projeto, o uso de detach() é aceitável, e a mitigação é feita ao fechar os sockets dos clientes na função parar(), o que força a thread de cliente a sair de seu loop de recebimento.

Diagrama de sequência cliente-servidor.



Conclusão

O sistema implementa os requisitos pedidos: servidor TCP concorrente, threads por cliente, broadcast de mensagens, logging thread-safe, proteção das estruturas compartilhadas e documentação. A análise com LLMs adicionou valor ao identificar riscos e opções de melhoria. Para entrega, o repositório contém código, scripts de teste, e o relatório completo.